3D Printer Converted Autonomous Chess-Playing Robot

Arne Jacobs, Sai Neha Narendra Babu, Christian Lazaro, Daniel Mcgurk, Vivek Devarapalli
{aj4023, sn4068, cal4000, dm2119, vd4009}@hw.ac.uk
Supervised by: Xianwen Kong & Theodore Lim

1 Abstract

This report presents the design, prototyping, and evaluation of an autonomous chess-playing robot built by repurposing a Creality Ender 3 Pro Cartesian 3D printer frame. The system integrates computer vision (OpenCV) for board-state recognition, the Sunfish Python chess engine for move generation, G-code motion control via OctoPrint, and an electromagnet end-effector for piece manipulation. The robot detects the human player's move through a top-mounted camera and responds autonomously by computing and executing its own move. The final prototype demonstrates full end-to-end autonomous gameplay on a standard chessboard.

2 Introduction

In the modern era of robotics, the boundary between specialised industrial tools and general-purpose automation is rapidly blurring. While 3D printers are traditionally viewed as manufacturing devices, their high-precision Cartesian motion systems provide an ideal foundation for complex pick-and-place applications. This project transforms a Creality Ender 3 Pro into an autonomous chess-playing robot, demonstrating how hobbyist-grade CNC hardware can be upcycled into an advanced robotic system through integration of computer vision and strategic AI, without expensive, bespoke hardware.

The robot uses a top-mounted camera and OpenCV-based image processing to detect the human player's move, the Sunfish chess engine to compute a response, and G-code motion control via OctoPrint to physically execute the move using an electromagnet end-effector. This report details the system architecture, design decisions, software pipeline, hardware modifications, testing outcomes, and reflections on the development process. The project repository is available at: GitHub Link and a demonstration video at: Youtube Link.

3 Literature Review

The development of an autonomous chess-playing robot draws upon several established fields of research.

3.1 Cartesian Robotic Manipulation

Cartesian robotic systems have long been favoured for pick-and-place tasks due to their mechanical simplicity and straightforward motion planning. Siciliano et al. [1] provide a comprehensive treatment of Cartesian kinematics, noting that the decoupled, independent-axis structure eliminates the need for inverse kinematic solvers required by articulated or SCARA configurations. The repurposing of consumer-grade CNC hardware as robotic platforms has gained traction in the maker and rapid-prototyping communities. Studies demonstrate that FDM 3D printer frames provide sufficient positional repeatability for manipulation tasks at low cost [2]. A directly relevant prior implementation was identified by Matou [3], who similarly repurposed a Cartesian 3D printer frame as a chess-playing robot. His project served as a baseline reference for the software architecture, with the present work extending and significantly modifying the codebase to incorporate a revised vision pipeline, hard-coded grid calibration, and improved move detection logic.

3.2 Computer Vision

The use of overhead cameras and image processing pipelines for board game state recognition is a well-studied problem. Bradski [4] established OpenCV as the standard library for such tasks, providing robust implementations of perspective correction, contour detection, and colour thresholding. Homography-based board rectification is a standard technique for recovering a top-down view from an oblique camera position. The core challenge of distinguishing occupied from unoccupied cells under variable lighting has been identified consistently in the literature as the primary source of error in vision-based board state systems, with thresholding fragility under non-uniform illumination being the most commonly reported failure mode [4].

3.3 Chess Engines and Move Generation

The algorithmic foundations of computer chess were established by Shannon [5], who formalised the minimax search tree as the basis for machine move selection. Shannon's framework, extended with alpha-beta pruning to reduce the effective branching fac-

tor, remains the computational core of most practical chess engines today, from lightweight implementations to full engines such as Stockfish [6].

3.4 Electromagnetic End-Effectors

Electromagnetic end-effectors are particularly suitable for ferromagnetic workpieces, offering reliable pick-and-release cycles without the mechanical complexity of tendon or pneumatic actuators [1]. Also, the electromagnet is powered only during piece transit. This energise-to-hold configuration is a common design pattern for minimising heat generation and power consumption in low-duty-cycle manipulation tasks.

4 Mechanism Selection & Conceptual Design

4.1 Platform Architecture

Three candidate robot architectures were considered for the manipulation platform:

Custom articulated arm (5–6 DOF): A serial chain of rotary joints offering maximum workspace flexibility and dexterity. However, this introduces significant inverse kinematics complexity and requires substantial fabrication and calibration effort.

SCARA robot: Four-axis architecture with two parallel rotary joints and one vertical linear axis, well-suited to fast, flat-plane pick-and-place tasks with relatively simple kinematics. Less mechanically complex than a full articulated arm but still requires custom fabrication.

Cartesian robot: Three independent linear axes (X, Y, Z) operating in a rectilinear workspace. Kinematics reduce to direct coordinate mapping with no IK solver required, making motion planning straightforward [1]. Mechanically simple and highly repeatable for structured, grid-based tasks such as chess piece manipulation.

A Cartesian architecture was selected as the best fit for this application: the chessboard is a regular grid perfectly aligned with a rectilinear workspace, piece manipulation requires only planar motion with vertical pick-and-place, and the simplified kinematics allowed development effort to be focused on the vision pipeline and software integration. Conveniently, a Creality Ender 3 Pro was offered to us by Dr. Hammer from the GRID. This Cartesian 3D printer with a 235 mm $\times$ 235 mm $\times$ 250 mm build volume exactly sufficient to house a standard chessboardbecame the natural choice for this project.

4.2 End-Effector Selection

Three end-effector concepts were evaluated:

Mechanical gripper: Requires custom fabrication and introduces gripping force control complexity. The close piece spacing on the board further limits the available gripping clearance, making reliable engagement difficult.

Vacuum gripper: Eliminates the clearance problem by approaching pieces from above. However, the small top surface of the 3D-printed chess pieces ($\sim$20 mm diameter) makes achieving a consistent airtight seal unreliable, leading to suction failure under slight misalignment.

Electromagnet: Approaches pieces from above without requiring an airtight seal, making it tolerant of positional offsets. Holding force scales with proximity rather than requiring perfect contact, and the electromagnet can be powered directly from the printer's existing fan output circuit with a simple enable/disable signal — requiring no additional hardware.

The electromagnet was selected for its positional tolerance, mechanical simplicity, and direct compatibility with the existing printer electronics.

5 Kinematic Modelling

5.1 Cartesian Coordinate System

The Creality Ender 3 Pro operates on a Cartesian coordinate system with three independent linear axes (X, Y, Z), each driven by stepper motors. The kinematic model is straightforward: each axis moves independently and the workspace is a rectilinear volume. This reduces inverse kinematics to trivial direct coordinate mapping:

$$P_{machine} = P_{board\_origin} + \begin{pmatrix} x_{square} \\ y_{square} \\ z_{transit} \end{pmatrix} \quad (1)$$

where P_{board_origin} is the calibrated offset of the board's A1 corner in machine coordinates, and (x_{square}, y_{square}) are computed from the square index and measured square pitch (approximately 27.5 mm per square for the scaled board).

5.2 Workspace Analysis

The Ender 3 Pro's build volume is 235 mm (X) $\times$ 235 mm (Y) $\times$ 250 mm (Z). But the workspace is

sligthly bigger as the tool can actually move out of the plates boundaries. This capacity is used for calibration and piece capture. The chessboard only occupies a 215 mm $\times$ 215 mm footprint leaving some space on two sides off the plate. This space was initially supposed to hold the captured pieces. More detail will be given in the last improvement from section 12. The Z axis is used solely for moving at a safe height where there is no chance of hitting the other pieces and for pick and placing.

5.3 Stepper Motor Positioning

The Ender 3 Pro uses stepper motors with a native resolution of 80 steps/mm on X and Y axes (with 1/16 microstepping). This yields a theoretical positional resolution of 0.0125 mm, well within the tolerance required for chess square targeting (squares are 27.5 mm wide).

6 Actuator Selection & Torque Analysis

6.1 Stepper Motor Adequacy

The Ender 3 Pro's stock stepper motors (42-34 NEMA 17, rated at 0.4 Nm holding torque) were retained without modification. The modified toolhead (the electromagnet mount and RS PRO electromagnet) has an estimated mass of approximately 0.35 kg, which is comparable to or lighter than the original hotend assembly it replaced (typically 0.4–0.5 kg including the heater block, nozzle, and cooling fan). The motors were therefore already proven adequate for a heavier load under the printer's original operation, confirming no upgrade was necessary for the modified configuration. Conservative feed rates were additionally selected to reduce dynamic loads and prevent piece displacement due to inertial effects during transit.

6.2 Electromagnet Specification

The RS PRO electromagnet (Stock No. 739-3277) provides 150 N holding force at 24 V DC. The mass of a typical 3D-printed chess piece with screw and bolt is approximately 12 g, requiring a minimum holding force of approximately 0.12 N. The electromagnet therefore provides a holding force margin exceeding 1000$\times$ the piece weight, ensuring reliable pickup across all tested configurations.

However, the force exceeding 1000$\times$ the piece weight brought another concern. When testing the electromagnet on custom combinations of screws, bolts and washers, the first combinations sticked to the magnet even when it was turned off because they were to light. This made it clear that the screw needed to be big enough and made heavier with bolts or washers. The 3D printed pieces were built in function of the optimal combination and in a way that the screw with bolt could simply be screwed inside (see Figure 1).

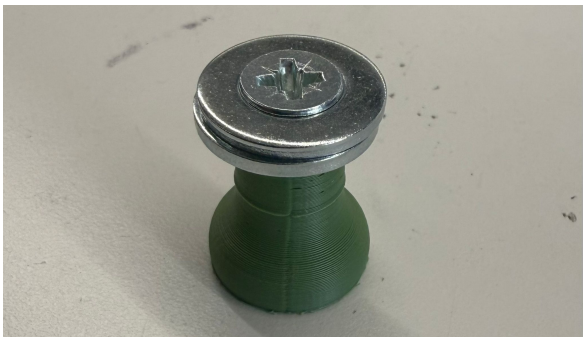

Figure 1: 1 screw, 2 large washers and 1 small washer screwed inside the 3D printed piece

7 System Design & Architecture

7.1 Overall Architecture

The system is structured as a sequential pipeline, illustrated conceptually in Figure 2

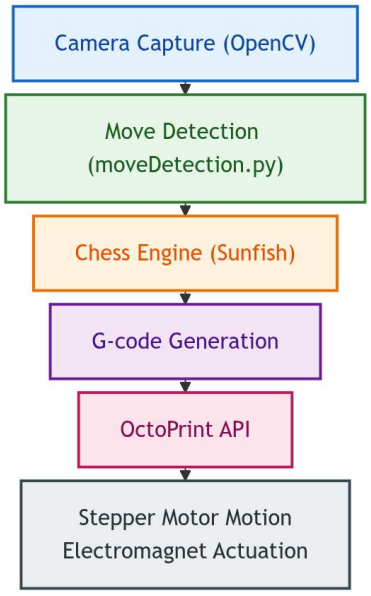

Figure 2: System Architecture

The software architecture builds upon the open-source baseline by Matou [3], substantially modified to incorporate the revised vision pipeline, hard-coded grid calibration, and improved move detection described in this report. All software components run on a laptop connected to the printer via USB. OctoPrint is used as the G-code intermediary, expos-

ing an HTTP API through which the Python control script sends motion commands. This decouples the chess logic from the printer firmware and allows rapid iteration of the control software without modifying firmware.

7.2 Mechanical Design

The Creality Ender 3 Pro frame was retained in its entirety. Modifications were limited to the toolhead: the original hot-end assembly was removed and replaced with a custom 3D-printed electromagnet mount as shown in Figure 3. The electromagnet is a DC solenoid type, powered via the original motherboard of the printer, connected through a circuit originally connected to the cooling fan for the hot-end. The mount was designed and 3D-printed in-house to position the electromagnet centrally and at a consistent, calibrated Z-height above the board surface, ensuring reliable pickup across all squares.

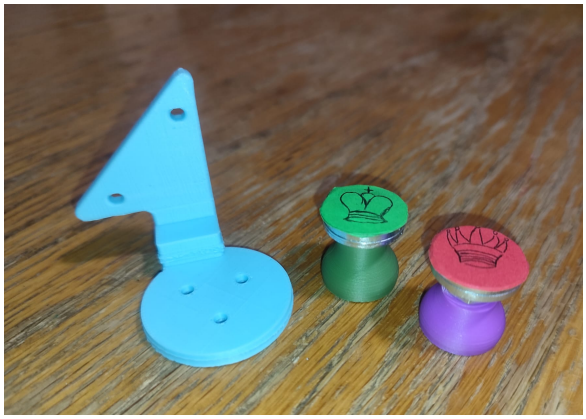

Figure 3: 3D printed components

In addition to the electromagnet mount, the chess pieces themselves were 3D-printed by the team. This allowed for precise control over piece geometry and mass, improving the consistency of electromagnet engagement and ensuring that piece dimensions were matched to the scaled board. Pieces were printed in two colours (green and pink filament) for human differentiation only. Indeed, the vision system doesn't detect pieces but only changes between a before and after image. Thus there is no need for colours. The metal part (screw + washers + bolt) was screwed on top so that the gripper could pick them up. For human differentiation green and red paper disks were glued on top of the pieces with drawings representing the pieces. The disk was made from paper in order to avoid interfering with the magnetic field.

The chessboard is a simple white paper with a hand-drawn 8×8 grid on it. It is fixed with Blu-Tack in a calibrated position relative to the printer coordinate

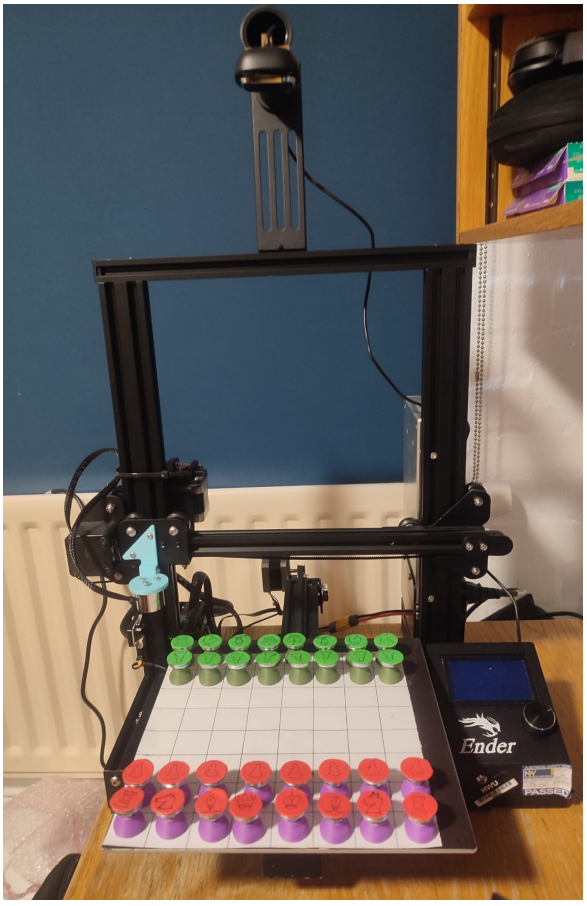

Figure 4: Creality Ender 3 Pro with electromagnet mount installed and chessboard with pieces

frame. The Blu-Tack makes it easy to remove in case it is decided to use a new board (e.g. a black and white variant or another size if needed for potential robot improvements).

7.3 Vision System

A camera is mounted above the board on a fixed overhead bracket attached to the printer frame, providing a top-down view of the entire board. OpenCV is used for all image processing. There are two core vision pipelines. The first one involves the following stages:

1. Image capture from the camera feed.
2. Detect the exterior lines of the game board using Hough.
3. Perspective correction using a homography transformation to obtain a top-down rectified view of the board.
4. Grid segmentation: dividing the rectified image into 64 equal-area regions corresponding to each square.

The program `calibrate_grid.py` permits the user to manually map the virtual and the real grid using the commands seen in Figure 5. These com-

```
--- GRID CALIBRATION ---
WASD : Move Grid
IKJL : Stretch/Shrink Grid
```

Figure 5: Commands shown in the terminal when manually calibrating the grid

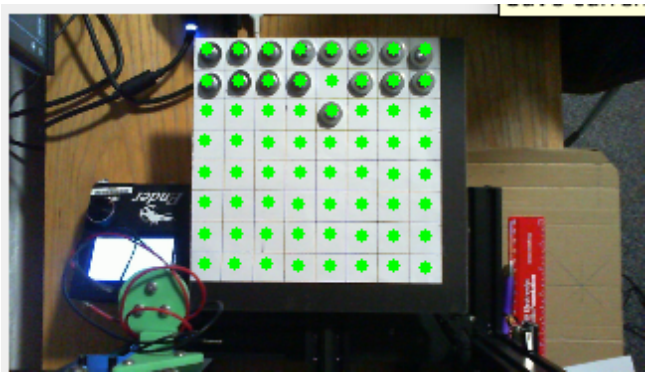

Figure 6: Grid on video for manual calibration

mands made it possible to move the virtual grid around and stretch it to adapt the light inclination of the camera. As shown in Figures 6, the calibration interface superposes an adjustable grid (the green dots) over the live camera feed, allowing the operator to visually align the grid to the physical board. Calibration parameters are then stored in `calibration_settings.json` and `grid_config.json` for persistence across sessions.

This calibration replaces the first vision pipeline but when the code does not detect the .json files it will automatically use the first vision pipeline.

The second core vision pipeline detects the human opponent's move and follows the following logic from `moveDetection.py`:

1. **Baseline capture:** An initial frame is captured and the pixel patch around each of the 64 square centres is stored as the reference board state.
2. **Motion detection:** Successive frames are compared to the previous frame using absolute pixel difference. The mean difference is compared against a pre-estimated background noise level. Only if this difference exceeds a significance threshold of 2.0 is a move flagged as in progress.
3. **Stillness detection:** Once a move has been flagged, the system waits until the scene has been still for 30 consecutive frames, ensuring the human has fully completed their move before analysis begins. If the scene never becomes still, no move is registered.
4. **Change localisation:** Each square's current pixel patch is compared against its baseline. The variance of the mean absolute difference is computed per square, and the two squares showing the greatest change are identified as the move candidates.
5. **Source/destination inference:** To distinguish the source square (now empty) from the destination square (now occupied), the mean pixel brightness of the two candidate squares is compared. The darker square is inferred to be the destination, as an occupied square appears darker under the overhead lighting.

When the move gets detected a before and after comparison image appears on the computers screen (see Figures 7 and 8). This was mainly used for implementing and troubleshooting the program but can be removed when simply playing.

Figure 7: This shows the system detecting that the player made his move

Figure 8: This shows the system detecting that the player captures a piece of the robot

Component	Specification	Value	Qty	Source / Cost
Creality Ender 3 Pro	FDM 3D printer, 220x220x250mm build vol., Cartesian XY, stepper motors, GRBL-compatible firmware	£180	1	Borrowed: GRID lab (£0)
RS PRO Electromagnet	RS Stock No. 739-3277, 150N holding force, 24V DC, 25mm dia., IP20, energise-to-hold	£25	1	Borrowed: JN Mechanical Workshop (£0)
Microsoft LifeCam HD-3000	720p HD webcam, USB, 30fps, wide-angle lens; used as overhead board camera	£30	1	Donated by lab assistant (£0)
M3/M4 Screw Kit	Assorted M3/M4 hex bolts, nuts and washers for frame mounting and electromagnet fixture	£8	1 set	Lab stock (£0)
PLA Filament (x3 colours)	Standard 1.75mm PLA (green, purple, blue) for chess pieces and electromagnet mount	£2	100g	Lab 3D printer filament stock (£0)
Laptop (control PC)	Existing team laptop; runs Python, OctoPrint, OpenCV	—	1	Personal equipment (£0)
	Total Market Value:	**£245**	**Total Actual Cost: £0 / £50 budget**	

Table 1: Component Specifications and Project Costs

7.4 Bill of Materials & Cost Analysis

All major components were borrowed from university facilities, resulting in zero direct expenditure against a £50 budget; the equivalent market value is approximately £245, as detailed in Table 1.

7.5 Chess Engine Integration

Sunfish [7], a compact open-source Python chess engine, is used for move generation. Sunfish represents the board state as a string and implements a minimax search with alpha-beta pruning. Given the current board state (updated after each detected human move), Sunfish returns the engine's chosen move in algebraic notation. This is then converted into source and destination square coordinates for the motion planner.

The interface layer (main.py) handles the full game loop: initialising the board, calling the vision system after each human move, passing the updated state to Sunfish, receiving the engine move, and dispatching motion commands to the Creality Ender 3 Pro.

7.6 Motion Control & OctoPrint Interface

Motion commands are issued as G-code strings sent to OctoPrint's REST API (octoprint.py). A typical move sequence comprises: see Figure 10.

Feed rates are set conservatively to avoid piece displacement from inertial effects. The G-code dwell command (G4) is used to ensure the electromagnet has time to energise or de-energise before motion resumes. Captured pieces are moved to a designated off-board holding area before the main piece is moved, to avoid collisions.

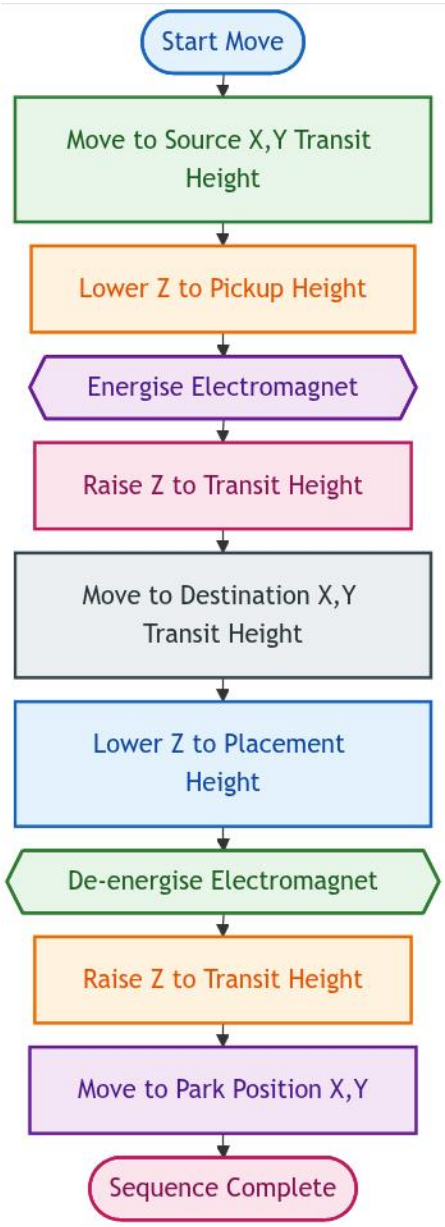

Figure 10: Typical move sequence

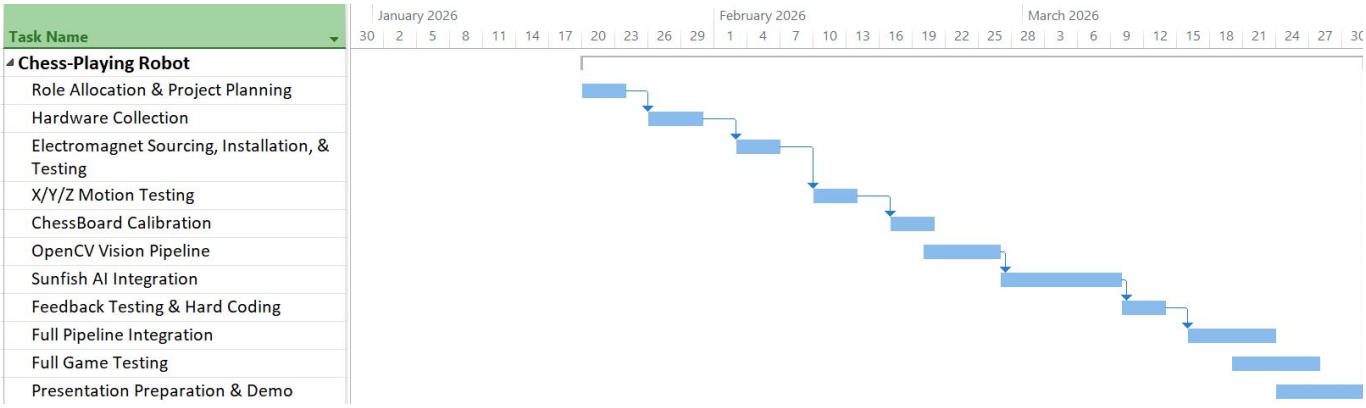

Figure 9: Gantt Chart

8 Prototyping, Testing & Engineering Practice

8.1 Iterative Development

Development proceeded in weekly iterations tracked in the project Gantt Chart from Figure 9. The initial weeks focused on project/team planning, hardware collection, and subsequent mechanical modifications to the 3D-printer via the replacement of the hot-end with an electromagnet fixed with a fabricated mount. Basic G-code motion tests were then calibrated and conducted. Vision pipeline development followed by chess engine integration were conducted in near-parallel once mechanical reliability was established. Full integration testing of the complete end-to-end pipeline was conducted in the final weeks before presentation.

8.2 Grid Calibration

The first vision pipeline seen in Section 7.3 worked correctly when testing in an environment with low ambient brightness. However, when changing location the first challenge appeared. The different lighting would often impact the Hough line detection and sabotage the grid. Indeed, since the board is composed of 9 lines in each direction, a difference of lighting quickly changes the focus from one line unto another. Thus, in some places the algorithm perfectly detects the outer lines but in other cases it misses them.

Another problem is where the light is coming from, when the light arrives sideways it creates shadow in the board which could also be assimilated to a “line". After multiple hours of fine-tuning without success in the GRID where it is very bright, it was decided to hard-code it.

In many cases, hard-coding is not the ideal option but in this project, the grid never changes, thus the position of the grid is always exactly the same for the camera, making it a perfect solution for prototyping. As it made sure that the grid was always adapted to the board it offered stability with testing later on.

8.3 Move Detection

Once equipped with a working grid it was time to start detecting the player’s moves. Once again the lighting caused problems. At first, the detection simply started at the slightest movement. The minimum variance required for the program to think the player was moving a piece was low enough for it to trigger without anything happening. The simple variance in ambient light without any movement occuring was already enough for it to think that something happened. With some fine-tuning it became possible to only start detecting when a big change in variance happens.

After this step the most frustrating and time consuming challenge became clear. To detect a move the program looks at a before and an after image and compares them. By detecting the two squares that show the biggest variance it becomes possible to identify what happened. At first, the pieces simply consisted of the screw, washers and bolt. All of these are highly reflective which made it hard to have a stable move detection. Indeed, in the grid where the ambient light is high both the white paper grid and the pieces would emit lots of light. Because of this and the influence of shadows the detection would often happen on squares where literally nothing happened.

After a while it became clear that when moving a piece the arm would cast a shadow on the board and make some cells darker. These cells got then detected instead of the one where pieces moved. By adding a wait system where the program waits for 30 still frames it became possible to remove shadow detection as the arm would already be gone for some time. This partially solved the problem and the move detection became a bit better but not good enough.

The average background noise was hardcoded and the high ambient light clearly surpassed that amount of noise. Thus before doing anything a noise detection was added. The system measures the average background pixel variation over 10 frames under the current lighting. This adaptive baseline means the motion detection threshold adjusts to whatever lighting conditions exist at startup rather than using a hardcoded value.

Also, **cv2.absdiff** was added to compare the current and baseline frames rather than comparing against a fixed colour or brightness value. This means gradual lighting changes during a game have less impact since both frames are subject to the same ambient light.

The last but most impactful decision in the code was the modification of the change metric. Rather than raw pixel values, the change metric per square is the variance of the mean absolute difference. This is somewhat robust to uniform lighting shifts across a square, since a uniform brightness change would affect the mean but not the variance much. Thanks to this the detection got a big improvement. Although far from good enough it would detect pieces about 80% of the time. Indeed, the metal pieces reflecting light still presented a problem for stable detection.

However, after all these hours spent on move detection the solution suddenly became obvious. By glueing less reflective paper on the metal parts the light reflected less and the difference between the originally white cell and the coloured cell after the move would become much clearer. The paper disks served a dual purpose: beyond reducing reflections, the distinct colours (green for the robot's pieces and red for the human's) created high-contrast, colour-distinct targets that made the variance comparison between the before and after image significantly more reliable, as the change signal on the destination square became much stronger and more consistent than with the bare metal pieces.

8.4 Testing Results

End-to-end testing was conducted over multiple complete game sequences in weeks ten through twelve. After fine-tuning some minor details over these two weeks the system achieved reliable end-to-end operation. It became possible to play entire games without any major issues. Once the system successfully demonstrated full autonomous gameplay it was time to start testing all components over time to analyse their success rates. Week 13 was used for extensive testing.

8.4.1 Piece Handling

300 out of 300 attempts were correctly picked up (see Figure 11), though upon placement pieces occasionally had been moved slightly out of their assigned cells over subsequent movements and placed too close to adjacent pieces. This occurrence primarily arose due to the electromagnet not having a uniform magnetic field. Thus, the pieces would sometimes by moved by a millimetre when being picked up. They would also very slightly move as the TCP positioning drifts after extended operation due to stepper backlash. However, due to the robot calibrating prior to each game, this gradual deviance has no impact, with the exception of very long games. Recalibration may be necessary after every 200 moves from the robot.

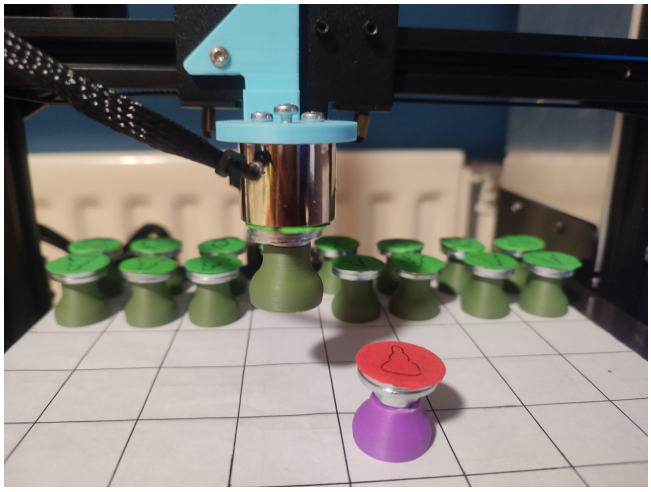

Figure 11: Robot executing a move: electromagnet engaged, piece in transit to target square

After all those tests, the average of pick and placing was 19.8 seconds, encompassing camera capture, OpenCV processing, Sunfish computation, G-code transmission, and physical execution.

As for the captured piece handling (see Figure 12): all of them were correctly relocated to the off-board holding area without collision in all tested scenarios (48 times). Over those 48 cases an average of 8.4 seconds was noted for the piece removal.

8.4.2 Move Detection

While conducting the 300 tests, movement was correctly detected only 267 times. However, some of the primary reasons for this error were noticed. First, if a piece is moved by the human opponent from behind the chess board, the camera doesn't detect enough movement and will not begin looking for a move. Additionally, lighting originating from sideways angles relative to the chess board cast shadows that cause the robot to interpret detected cells as unchanged.

The last 80 tests were conducted somewhere without

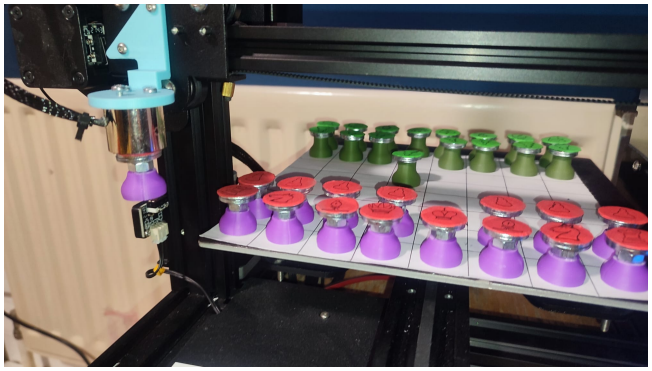

Figure 12: Robot removing a piece it is capturing from the board before placing its own piece instead

sideways lighting and while taking care of not making small piece movements from behind the board. Out of the 80 tests only one move was wrongly detected which was a serious improvement on the first 220 tests (14.5% vs 1.25%).

It is important to note that incorrect detections mainly lead the system to believe an illegal move has been made and therefore does not accept it. This leaves the possibility to move the piece back and try removing the piece again in the hopes that the system correctly detects it. However, as the game develops, moments occur where multiple movements are possible, thus leaving the system more susceptible to accepting a wrongly interpreted opponent movement. Therefore, a “back" button allowing for rewinding the most recently detected opponent movement may be necessary for future works towards this project.

9 Project Management, Teamwork & Communication

The team of five divided responsibilities across mechanical, electrical, vision, software, and integration domains, with a team leader elected in Week 2. Development followed weekly iterations tracked in the project annex (Table 3). Communication was maintained via WhatsApp and weekly in-person meetings. On Wednesdays afternoon the team met for 2 hours and on Fridays during the 1 hour implemented in the RMS course.

10 Ethics, Sustainability, Security & EDI

10.1 Sustainability

A central sustainability feature of this project is the deliberate reuse of an existing commercial platform (Ender 3) rather than designing a bespoke robot from scratch. This approach reduces material consumption, embodied energy, and manufacturing waste. The use of open-source software (Sunfish, OctoPrint, OpenCV) eliminates licensing costs and supports the broader open hardware and software ecosystem.

The electromagnet end-effector consumes power only during piece manipulation (active low duty cycle), minimising energy use and the software runs on an existing laptop rather than requiring dedicated embedded hardware.

10.2 Safety

The system operates at low speeds with conservative feed rates to prevent unexpected or forceful motion. To validate the safety of the Z-axis motion, the maximum downward force was assessed by manually resisting the toolhead descent at the configured feed rate. The stepper motor skipped steps under light hand resistance, confirming that the axis cannot exert sufficient force to cause injury before stalling. Combined with the conservative feed rates and the constrained build volume preventing unexpected motion outside the operational area, the system presents no meaningful crush hazard in an office or educational environment.

10.3 Accessibility & EDI

Chess is a game with significant cognitive accessibility considerations. An autonomous robot opponent removes the need for a second human player, enabling solo practice for players. Currently, the Sunfish engine is set to only play the most efficient moves possible for the engine which plays at an ELO of 2000 on Lichess.org. This is generally higher than 95% of chess-players thus not very accessible for most players. It’s difficulty could in principle be adjusted to accommodate different skill levels in the future, improving inclusivity. The physical interface (placing pieces on a board) is standard and does not impose additional accessibility barriers beyond those of chess itself. However, for improved accessibility to chess itself, a speech detection system could be implemented in future works, wherein the system moves even the human opponent’s pieces for them, in the case of a human opponent unable to do so themselves.

10.4 Security

The OctoPrint API used for motion control operates on a local network. In the current prototype configuration, the API is accessible without authentication

on the local host, which is acceptable for a laboratory prototype but would require authentication and network isolation in any deployed context. No personally identifiable data is collected or stored by the system.

11 Discussion

The prototype met all four primary objectives: board-state perception, move computation, motion planning, and physical manipulation. The 100% piece pickup rate demonstrates that the Cartesian platform and electromagnet end-effector are well-matched to the task. However, the 89% move detection rate represents the system's principal limitation, falling short of a robust deployment standard. Even though the detection can reach 98.75%, the fact that it depends on lighting and how the player moves the piece is still very limiting.

The 20-second average move cycle is longer than comparable vision-integrated robotic systems reported in the literature, primarily due to the low maximum speed achievable on the Z-axis because of the lead screw. But these lower speeds made the process more reliable and safer. The Sunfish engine's reduced playing strength relative to full engines such as Stockfish is an acknowledged trade-off; in a deployment context, difficulty selection would allow the engine to be tuned to the opponent's skill level [6].

12 Future Improvements

Several extensions would improve the system substantially:

UI controls: A “back" button to rewind incorrectly detected moves, and a “pause" button to allow human correction, would mitigate the impact of the 11% detection failure rate without requiring a full game restart.

Adaptive difficulty: Integrating Elo-rated play via Sunfish's configurable search depth would improve inclusivity for beginners.

Lighting control: A diffuse ring light mounted on the printer frame above the board would eliminate shadow and specular reflection artefacts, potentially recovering the full detection accuracy observed under controlled laboratory conditions. This would make it possible to reliably return to an automatic grid detection.

Voice control: Microphone integration with a speech-to-move parser would improve accessibility for players with limited hand mobility. This option would also enable players to play against each other using only voice commands. The Robot would then move the pieces for both players.

Persistent game state: Saving board configurations to disk would allow interrupted games to be resumed across sessions.

Display screen reuse: Controlling the printer's small screen next to the board would enable to do all the above directly through the display and it's button. This could help select settings to choose the ELO-difficulty, which sides starts playing, to save the game, etc.

Automatic reset: This is certainly the hardest requiring for more complex machine vision. The idea would be to use the free sides of the plate (the strokes where there is no board) as a zone for captured pieces. By placing all the captured pieces there (still accessible by the gripper) it would be conceivable to have to robot reset all the pieces before starting a new game.

13 Conclusion

This project successfully demonstrated the conversion of a Creality Ender 3 Pro 3D printer into a fully autonomous chess-playing robot. The system integrates computer vision for board-state perception, the Sunfish chess engine for move computation, G-code motion control via OctoPrint, and an electromagnet end-effector for piece manipulation, achieving complete end-to-end autonomous gameplay without human intervention.

The primary technical challenge, reliable grid calibration under variable lighting, was resolved through a hard-coded calibration approach, enabling stable operation under controlled conditions. Physical piece manipulation achieved a 100% pickup success rate over 300 attempts, while move detection achieved 89% accuracy (267/300), with identified failure modes attributable to camera occlusion and shadow interference rather than fundamental algorithmic limitations.

The project demonstrates that low-cost Cartesian platforms can be effectively repurposed for robotic manipulation tasks when combined with appropriate sensing, planning, and control software. All project objectives were met within the £50 budget constraint through systematic resource reuse, with a market-equivalent system valued at approximately £245.

References

[1] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*, ser. Advanced Textbooks in Control and Signal Processing. Springer London, 2009. [Online]. Available: https://link.springer.com/book/10.1007/978-1-84628-642-1

[2] R. Siegwart, I. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots, second edition*, ser. Intelligent Robotics and Autonomous Agents series. MIT Press, 2011. [Online]. Available: https://books.google.co.uk/books?id=4of6AQAAQBAJ

[3] Matou, "3d printer chess," https://github.com/matou/3d-printer-chess, 2020, accessed: April 2026.

[4] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000. [Online]. Available: https://www.researchgate.net/publication/233950935_The_Opencv_Library

[5] C. E. Shannon, "Xxii. programming a computer for playing chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 41, no. 314, pp. 256–275, 1950. [Online]. Available: https://doi.org/10.1080/14786445008521796

[6] T. Romstad, M. Costalba, J. Kiiski, G. Linscott *et al.*, "Stockfish: A strong open source chess engine," https://stockfishchess.org/about/, 2008.

[7] T. Ahle, "Sunfish: A simple, but strong chess engine in Python," https://github.com/thomasahle/sunfish, 2013.

14 Appendices

14.1 Team Contribution

Member	Primary Responsibilities	Weeks Active
Arne Jacobs	Research for related work. Electromagnet mount fabrication, chess pieces assembly, computer vision pipeline (`moveDetection.py`, `calibrate_grid.py`), chess engine integration (Sunfish), game logic (`main.py`), OctoPrint interaction (octoprint.py), hardware testing, final product testing and tweaking. Presentation and report.	2–14
Sai Neha Narendra Babu	Research for related work. Initial set up of the Github project with first versions of the important scripts (`main.py`, `sunfish` and `MoveDetection.py`). Presentation and report.	2–14
Vivek Devarapalli	Technical advisory on hardware testing.	2–12
Daniel McGurk	Chess piece design.	4–12
Christian Lazaro	Research for related work. Testing, documentation, project/team management. Presentation and report.	2–14

Table 2: Summary of Team Member Contributions

14.2 Weekly Progress

Week	Key Activities	Outcome / Evidence
2	Team leader election; brief review; role allocation; initial project planning	Roles assigned; Gantt chart drafted
3	Hardware collection (Arduino, Ender 3); disassembly of hotend; assessment of frame	Printer prepared; Bill of materials finalised
4	Electromagnet sourced and tested; custom mount designed; fabrication begun	Mount CAD model; electromagnet continuity test
5	Mount fabricated; X/Y/Z motion tests via G-code; coordinate frame established	G-code test scripts; video of axis motion
6	Chessboard fixed in frame; square coordinate mapping generated; `calibrate_grid.py` v1	`calibration_settings.json`; grid overlay image
7	OpenCV pipeline; `moveDetection.py` initial version	Detection accuracy estimated at 80% in lab lighting
8	Sunfish integration; full move generation loop tested	Move detected and associated with a chess move
9	Feedback session; lighting issues identified; calibration hard-coding decided	Updated `grid_config.json`; feedback notes
10	Hard-coded calibration implemented; full pipeline integration; OctoPrint API tested	Successful pick-and-place
11	Bug fixes in move sequencing; captured piece logic added	Video showing move detection, pick and place and peice capture
12	Presentation preparation; final testing; demonstration	Presentation delivered; prototype demonstrated
13	300 additional tests for accurate measurements	Detection accuracy: 89% (98.75% in good conditions), Pick&PLace succes rate: 100%
14	Report writing	Report delivered

Table 3: Weekly Progress